



TECNOLÓGICO
NACIONAL DE MÉXICO

TECNOLOGICO DE ESTUDIOS

SUPERIORES DE ECATEPEC

TESEhambreados

Fundamentos de Ingeniería de Software

Juan Alberto Flores González

Proyecto de Ingeniería de Software

Integrantes

Calderón Garcia Dana Patricia

Flores Godinez Antonio

Guerrero Alonso Axel

Munguía Cardenas Marco Agustin

Ortega Cruz Cristian Abraham

Prado Bravo Alan Andrés

Fecha de entrega: 12 de Octubre 2026

Sistema móvil para la gestión de pedidos de alimentos dentro del TESE · Fundamentos de Ingeniería de Software · Noviembre 2026

Este reporte se basa en el documento *Copia de TESEhambreadosdoc1.5.docx* (entrega del 12 de octubre de 2026) y en la actividad del Tema 4 (Modelo de Diseño).

1. Descripción general

TESEhambreados es una aplicación móvil desarrollada con Flutter y Dart, que usa Firebase y Cloud Firestore como base de datos en la nube. Permite a los estudiantes del Tecnológico de Estudios Superiores de Ecatepec consultar los establecimientos de alimentos (cafetería y quiosquitos), revisar menús, precios y disponibilidad, y hacer pedidos anticipados para recogerlos en el local. El pago se hace físicamente, por lo que la app no maneja pagos electrónicos. Hay dos perfiles: el estudiante y el personal del establecimiento.

2. Justificación

Los estudiantes compran alimentos sobre todo en recesos o entre clases, con poco tiempo, y se encuentran con filas y productos agotados. La encuesta aplicada en Google Forms respalda el problema:

- 76.5 % compra durante su tiempo libre.
- 64.8 % dispone de menos de 15 minutos (32.4 % menos de 10 y 32.4 % de 10 a 15).
- 41.2 % tarda entre 5 y 10 minutos en llegar al establecimiento.
- 44.1 % encuentra filas con frecuencia.
- 61.8 % algunas veces no encuentra el producto que busca.
- 82.4 % necesita conocer el precio antes de comprar.
- 70.6 % usa algún establecimiento del TESE durante la semana.

Estos datos justifican tres funciones centrales: catálogo con precios visibles, indicador de disponibilidad y pedido anticipado con seguimiento de estado.

3. Objetivos

Objetivo general. Desarrollar e implementar una aplicación móvil para gestionar y consultar pedidos de alimentos dentro del TESE, que permita ver establecimientos, menús, precios y disponibilidad, y generar pedidos de forma anticipada para agilizar la compra y reducir tiempos de espera.

Objetivos específicos (resumen). Analizar requisitos mediante cuestionarios; diseñar la interfaz en Figma; diseñar la base de datos; desarrollar la app en Flutter; implementar los módulos de consulta de menús, selección de productos y creación de pedidos, y registro y consulta del estado de pedidos; realizar pruebas funcionales; integrar los módulos; y documentar desarrollo, pruebas y mantenimiento.

4. Lo que ya se ha hecho

Área	Avance documentado en la versión 1.5
Planificación	Metodología incremental, cronograma de 15 actividades (9 sep–2 dic 2026) y tabla PERT-CPM con ruta crítica.

Área	Avance documentado en la versión 1.5
Investigación de campo	Encuesta de 16 preguntas, evidencia con capturas del formulario y tabla de análisis con 14 aspectos interpretados.
Ingeniería de requisitos	Cinco etapas definidas y 8 requisitos (RF-01 a RF-07 y RNF-01).
Arquitectura	Tres capas (presentación, lógica y datos) con flujo general de arquitectura.
Base de datos	Colecciones Usuarios, Locales, Menu_Platillos, Pedidos y Resenas con campos en minúsculas (uid, id_local, estado_orden…), estados del pedido 0 a 3, artículos del pedido como arreglo de objetos, diccionario de datos, modelo lógico, modelo entidad-relación y normalización 1FN–3FN.
Interfaz	15 pantallas (UI-01 a UI-15) en tres módulos: autenticación, estudiante y administrativo, con Material Design 3.
Firebase	Proyecto tese-hambreados creado, Firestore en modo de prueba en us-central, app Android registrada, `google-services.json`, dependencias `firebase_core` y `cloud_firestore`, permiso de internet, inicialización en `main.dart` y sincronización con streams.
Catálogo	30 productos de ejemplo (8 desayunos, 10 comidas, 8 bebidas y 4 postres) con precio, tiempo e ingredientes.
Flujo	Diagrama de flujo de actividad para estudiante y personal.

Según el cronograma, al 9 de noviembre el proyecto está en la fase de codificación (A10, hasta el 13 de nov.): el análisis y el diseño están documentados y falta cerrar el desarrollo.

5. Lo que falta por hacer

1. **Terminar la codificación (A10)** de todos los módulos: consulta de locales y menús, carrito, pedidos, seguimiento, panel del personal, gestión de menú y registro de productos.
2. **Integración de módulos (A11, 14–20 nov.)** para tener una versión funcional completa.
3. **Pruebas del sistema (A12, 21–27 nov.)**, tanto por módulo como de extremo a extremo.
4. **Correcciones y ajustes (A13, 28–30 nov.).**
5. **Documentación final (A14)**, incluyendo pruebas y mantenimiento, como pide el último objetivo específico.
6. **Entrega del proyecto (A15, 2 dic.).**
7. **Actividad del Tema 4:** estrategias de diseño (4.1), mínimo 6 clases con diagrama de clases (4.2) y diseño de sistema con flujo de extremo a extremo y riesgos (4.3).

Casi todas las actividades son críticas y A13 solo tiene 0.4 semanas de holgura, por lo que un retraso en la codificación mueve la entrega final.

6. Aspectos que pueden mejorarse

6.1 Planificación

- **Duración inconsistente.** El PERT calcula 14.2 semanas, pero del 9 de septiembre al 2 de diciembre hay unas 12 semanas. Además, A3 y A4 se indican como 2 semanas, aunque sus fechas (16–25 sep) abarcan 10 días.
- **Fórmula dañada.** La definición de holgura aparece como "IT−ICIT-ICIT−IC"; debe decir H = IT − IC.
- **Tabla del cronograma con una columna vacía** al final (sexta columna sin encabezado) que debe eliminarse.
- **Metodología vs. cronograma.** Se declara un proceso incremental, pero las fases se ejecutan en secuencia. Convendría definir incrementos concretos (catálogo, pedidos, panel del personal) con entregas parciales.
- Faltan **roles del equipo, riesgos del proyecto y plan de contingencia**.

6.2 Requisitos

- Solo hay **8 requisitos**. Faltan funcionales como registro e inicio de sesión, acceso por PIN del personal, historial, perfil, reseñas y cancelación de pedidos.
- Hay **un solo requisito no funcional**. Faltan rendimiento, seguridad, disponibilidad, compatibilidad con versiones de Android, usabilidad medible y escalabilidad.
- El propio documento los llama "propuesta inicial" y no muestra evidencia de **validación**. Conviene agregar una matriz de trazabilidad (necesidad → requisito → pantalla → prueba) y casos de uso o historias de usuario.
- La encuesta solo cubre a estudiantes; **no se consultó al personal de los locales**, de quienes depende actualizar menú, disponibilidad y estados.

6.3 Encuesta

- Los porcentajes (32.4 %, 29.4 %, 61.8 %…) corresponden a una muestra de unas 34 personas, pequeña para generalizar. Debe indicarse el tamaño de la muestra, cómo se eligió y sus límites.
- Las preguntas que aparecen como lista con viñetas deberían numerarse para poder citarlas en el análisis.
- Falta una pregunta sobre la disposición a usar la app y a pedir con anticipación.

6.4 Incongruencias entre secciones

- **Pago.** Se afirma que no habrá pagos electrónicos, pero UI-07 (Figura 7) habla de "selección del método de pago" y de "proceder con el pago", y la colección Pedidos incluye `metodo_pago`. Debe quedar como "pago en el establecimiento" o eliminarse el campo.
- **Campos diseñados vs. implementados.** El diseño usa `stock`, `calificacion_promedio`, `imagen_url` y `estado_orden`; el apartado de configuración usa `existencias`, `calificacion`, `numCalificaciones`, `fotos` y `estado`, además de nombres como `idPedido` y `horaLlegada`. El diccionario de datos debe actualizarse con lo realmente implementado (y mantener un solo estilo de nombres).
- **Estados del pedido.** El estado 1 aparece como "En preparación", "Preparando" y "Cocinando"; el 2 como "Listo para recoger" y "Listo". Se recomienda un solo nombre por estado.

- **Tabla de artículos del pedido.** La columna de tipo se llama "dato" en lugar de "Tipo de dato".
- **Categorías.** El catálogo alterna "Burritos y comidas" y "Comida". La colección Locales no tiene campo de ubicación, mientras que `carrera_area` se describe como ubicación del local.
- **Objetivo general vs. diseño.** Se habla de "programar" pedidos y ver información "en tiempo real", pero no hay hora de recogida programada en el modelo de datos (solo `horaLlegada` en la implementación). La actividad del Tema 4 también pide hora de recogida y notificaciones.
- **Reseñas.** Existen en la base de datos y en el modelo ER, pero no hay pantalla ni requisito para ellas entre las 15 interfaces. Lo mismo ocurre con las promociones de UI-05.
- **Numeración y redacción.** La sección de Firebase empieza en el punto 2 (falta el 1) y mezcla segunda persona ("tu proyecto"); el título "7. RELACIÓN ENTRE LA INVESTIGACIÓN Y EL SISTEMA" y "15. Formulario para registrar nuevos platillos" rompen el formato de las demás secciones y figuras. El nombre de la app aparece como "tese-hambriados" y el paquete como `com.example.tese_food_app` (nombre de ejemplo).

6.5 Base de datos y seguridad

- **Reglas de Firestore en modo de prueba.** Dejan la base abierta y el modo de prueba caduca; antes de la entrega hay que definir reglas por rol (el estudiante lee menús y administra sus pedidos; el personal solo su local).
- **Autenticación.** Se menciona Firebase Authentication, pero el personal entra con un PIN. Debe definirse dónde se guarda, evitando texto plano en el cliente y PIN compartidos.
- **Imágenes en Base64 en Firestore.** Es gratuito, pero cada documento está limitado a 1 MiB. Hay que comprimir las fotos y documentar el límite.
- **Control de existencias.** Dos estudiantes pueden pedir el último producto a la vez; se recomienda usar transacciones de Firestore.
- El ordenamiento en el cliente (`List.sort`) sirve con pocos datos; debe documentarse como decisión para el tamaño actual.
- Se mencionan WebSockets; Firestore maneja su propio canal en tiempo real, por lo que conviene hablar de *listeners/streams de Firestore*.

6.6 Funcionalidad y experiencia de uso

- **Notificaciones push** (Firebase Cloud Messaging) cuando el pedido esté "Listo para recoger".
- **Cancelación y pedidos no recogidos:** no se define qué pasa si el estudiante no llega o el local rechaza el pedido.
- **Límite de pedidos simultáneos** o aviso de saturación en horas pico.
- **Falta de conexión:** mensajes claros y uso de la persistencia local de Firestore.
- **Plataforma:** solo se configura Android; debe aclararse si iOS queda fuera del alcance.

6.7 Pruebas y evaluación del éxito

- No hay **plan de pruebas** (casos, datos, criterios de aceptación). Se sugiere una tabla por requisito, pruebas con 5 a 10 estudiantes y una prueba piloto en un local real.
- Faltan **indicadores de éxito**: reducción del tiempo de compra, porcentaje de pedidos entregados correctamente y satisfacción medida con una escala simple.
- Faltan **despliegue y mantenimiento**: quién actualiza el menú, cómo se distribuye la APK y quién administra Firebase tras la entrega.

7. Propuesta para la actividad del Tema 4 (Modelo de Diseño)

- **Estrategias de diseño:** diseño orientado a objetos y arquitectura por capas (presentación, lógica, datos), con enfoque centrado en el usuario para las interfaces.
- **Clases mínimas:** Usuario, Local, Producto (Platillo), Pedido, ItemPedido, HorarioRecogida, Notificacion, EstadoPedido y Resena. La clase Pago puede reducirse a registrar "pago en establecimiento".
- **Estados del pedido:** RECIBIDO, EN_PREPARACION, LISTO y ENTREGADO, que coinciden con los valores 0 a 3 de la base de datos.
- **Componentes:** cliente Flutter, Firestore, panel de cafetería y módulo de notificaciones; no aplican pasarela de pago ni API REST propia porque Firebase cumple el papel de backend.
- **Riesgos y decisiones:** límite de pedidos por franja horaria, tiempo de tolerancia para recoger, cancelación automática de pedidos no recogidos y persistencia local ante fallas de internet.

8. Conclusión

La versión 1.5 muestra un proyecto con base sólida: problema respaldado con datos, arquitectura clara, base de datos bien descrita, 15 interfaces y Firebase ya conectado. Para la etapa final conviene priorizar (1) terminar y probar el flujo principal de pedido, (2) corregir las incongruencias de documentación y formato, (3) asegurar las reglas de Firestore y (4) completar requisitos, plan de pruebas y la actividad del Modelo de Diseño, de modo que todo quede coherente para la entrega del 2 de diciembre.